

# SMART ATTENDANCE MANAGEMENT SYSTEM

## (SRS Report)

## JAMIA MILLIA ISLAMIA

Department of Computer Engineering  
Bachelor of Technology

Members:

- 1) Mohd Anas (24BCS061)
- 2) Saniya Parveen (24BCS072)

# 1. Introduction

## 1.1 Purpose

The purpose of this document is to define the functional and non-functional requirements for the BLE Proximity-Based Smart Attendance System. This system automates classroom attendance verification using Bluetooth Low Energy (BLE) proximity detection, server-issued ephemeral tokens, and strict device binding. The document serves as the primary reference for the development team, project supervisors, and stakeholders throughout the software development lifecycle.

## 1.2 Product Scope

The Smart Attendance Management System is a cross-platform mobile application suite designed for educational institutions. The system consists of two mobile applications: a Student App for broadcasting attendance signals and an Instructor App for scanning and collecting those signals. A backend server handles token generation, attendance validation, and academic management. An administrative interface manages curriculum mapping, instructor assignment, and device exception handling.

The system replaces manual roll-call processes with an automated, proximity-verified mechanism. Students broadcast a temporary, server-generated token via Bluetooth during a configurable 2 to 5-minute window. The instructor's device scans for these tokens, verifies physical proximity via signal strength, and submits a batch report to the server. The server validates tokens, marks attendance, and provides drill-down reporting for both instructors and students.

The system does not include facial recognition, GPS-based geofencing, or integration with third-party Learning Management Systems in the current release.

### 1.3 Technologies to be Used

| Layer              | Technology              | Purpose                                                    |
|--------------------|-------------------------|------------------------------------------------------------|
| Mobile Application | Flutter                 | Cross-platform development for Student and Instructor apps |
| BLE Communication  | flutter_blue_plus       | BLE scanning and advertising on mobile devices             |
| Backend Framework  | Node.js with Express.js | REST API development and request handling                  |
| Database           | PostgreSQL              | Relational data storage                                    |
| ORM                | Prisma                  | Database schema management and query execution             |
| Authentication     | JSON Web Tokens (JWT)   | Stateless API authentication                               |
| Password Hashing   | bcrypt                  | Secure password storage                                    |
| API Communication  | HTTPS / REST            | Secure data exchange between mobile and server             |

## 1.4 Definitions, Acronyms and Abbreviations

| Term            | Definition                                                                                                                            |
|-----------------|---------------------------------------------------------------------------------------------------------------------------------------|
| BLE             | Bluetooth Low Energy. A wireless communication protocol optimized for low power consumption.                                          |
| RSSI            | Received Signal Strength Indicator. Measures the power level of a received Bluetooth signal, used to determine physical proximity.    |
| UUID            | Universally Unique Identifier. A 128-bit label used for unique identification.                                                        |
| Ephemeral Token | A temporary UUID generated by the server, valid for 5 minutes, broadcasted via BLE.                                                   |
| Device Binding  | The process of mapping a student's account to a specific physical mobile device using an OS-level identifier.                         |
| GATT            | Generic Attribute Profile. The BLE protocol for data exchange between devices.                                                        |
| Peripheral Mode | BLE mode where a device broadcasts data. Used by the Student App.                                                                     |
| Central Mode    | BLE mode where a device scans for broadcasts. Used by the Instructor App.                                                             |
| ORM             | Object-Relational Mapping. A technique to convert data between incompatible type systems using object-oriented programming languages. |
| JWT             | JSON Web Token. A compact, URL-safe means of representing claims between two parties.                                                 |
| SRS             | Software Requirements Specification.                                                                                                  |
| DFD             | Data Flow Diagram.                                                                                                                    |
| ERD             | Entity-Relationship Diagram.                                                                                                          |

## 2. Overall Description

### 2.1 Product Perspective

The Smart Attendance System is a self-contained product that operates independently of existing university infrastructure. It does not require integration with the university's existing ERP or Learning Management System for core functionality. The system communicates internally between three components: the Student Mobile App, the Instructor Mobile App, and the Backend Server.

The system follows a centralized processing model. All attendance validation, token generation, and data persistence occurs on the backend server. The mobile applications serve as data collection and broadcast interfaces. The instructor's device acts as an intermediary between student BLE broadcasts and the server, reducing the need for continuous student internet connectivity.

### 2.2 User Classes and Characteristics

**Students:** The primary users. They tap one button in the mobile app to broadcast their BLE token during the attendance window and view their attendance history per subject. They require a simple interface with no manual Bluetooth configuration.

**Instructors:** They start the 2 to 5-minute attendance window, scan for student BLE tokens, submit the batch to the server, and view Present or Absent reports per session. They require a clear dashboard with live detection counts.

**Administrators:** They manage programs, curriculum mapping, instructor assignments, auto-enrollment, and device change approvals through a web dashboard. They ensure data integrity and smooth semester operation.

### 2.3 Operating Environment

The **mobile application environment** supports Android 8.0+ and iOS 13.0+, requires Bluetooth 4.0 (BLE), internet for authentication and sync, and at least 200MB storage.

The **server environment** uses Node.js (v18+), Express.js (4.x), PostgreSQL (v14+), and Prisma (v5.x), deployed on Linux (Ubuntu 22.04) or Windows Server.

The **development environment** includes Visual Studio Code, Git & GitHub for version control, Postman/Insomnia for API testing, and pgAdmin 4 for database management.

## 2.4 Software Interfaces

### Software Interfaces

- **Mobile-to-Server Interface:** the application communicates with the backend over HTTPS using JSON format. Authentication is handled via JWT Bearer tokens in the Authorization header, with `application/json` as the content type.
- **Mobile-to-Mobile Interface (BLE):** devices communicate using Bluetooth Low Energy advertising, with a maximum packet size of 31 bytes. A 128-bit UUID is broadcast as the service identifier, enabling detection within a 5–7 meter range (RSSI  $\approx$  -75 dBm).
- **Server-to-Database Interface:** the backend interacts with the PostgreSQL database using the Prisma Client over TCP/IP, enabling type-safe queries based on the defined schema.

## 2.5 Hardware Requirements

### Student Device:

- Smartphone with Bluetooth 4.0 or higher
- Android or iOS operating system
- Active internet connection (cellular data or Wi-Fi) for token retrieval

### Instructor Device:

- Smartphone with Bluetooth 4.0 or higher
- Android or iOS operating system
- Active internet connection for batch submission
- Sufficient battery capacity for 2 to 5 minutes of continuous BLE scanning

### Admin Device:

- Desktop having a Internet Browser . (Web interface)
- Active internet connection for assigning courses , approving request to server.

### Server Hardware:

- Minimum 2 CPU cores
- Minimum 4 GB RAM
- Minimum 20 GB SSD storage

Network interface with public IP or domain name for mobile app access

## 2.6 Functionalities

### User & Device Management

- **Authentication & Profile:** Secure management for students, instructors, and admins.
- **Device Binding:** Client binds accounts to hardware IDs; server validates these to prevent proxy attendance.

- **Device Changes:** Students request changes via the mobile client; admins approve them via the web dashboard.

### Attendance Session Management

- **Initiation:** Instructors start sessions on the client; the server generates a 2-5 minute ephemeral UUID.
- **Detection:** Instructor client broadcasts the UUID via BLE; student clients auto-detect and store it.
- **Submission:** Instructor client submits all detected UUIDs to the server in a single batch.
- **Validation:** Server verifies UUIDs, device bindings, and timestamps, rejecting invalid entries.

### Academic Management

- **Auto-Enrollment:** Server enrolls students in subjects based on predefined curriculum mappings.
- **Assignments:** Admins assign instructors to courses via the dashboard; instructors view them on the client.
- **Creating Course Structure:** Making new Courses and subjects in the courses.

### Reporting & Analytics

- **Client Views:** Instructors view session attendance; students view personal history and percentages.

**Admin Views:** Admins monitor institution-wide trends and review flagged anomalies via the dashboard.

## 2.7 General Constraints

- The BLE advertising packet is limited to 31 bytes. All broadcast data must fit within this constraint.
- The Flutter framework does not support Extended BLE Advertising natively. The system must use standard BLE advertising.
- Students require an active internet connection for exactly one HTTP request to obtain the ephemeral token before broadcasting.
- The system supports a maximum attendance window of 5 minutes per session.
- The RSSI threshold for proximity detection is fixed at -75 dBm.
- The system does not support offline attendance marking. All data must be synchronized with the server.
- The backend server must be accessible via HTTPS. Plain HTTP is not permitted.
- Passwords must be hashed using bcrypt. Plain-text password storage is prohibited.

## 3. System Features

### 3.1 Functional Requirements

#### A) Student Registration and Device Binding

- Students register using university credentials; first device is auto-approved and stored (ID, model, OS).
- System binds account to a single device and verifies it on login.
- Attendance features are blocked if the device ID does not match.

#### B) Attendance Token Generation

- System generates a unique 128-bit UUID with 5-minute validity.
- Token is linked to student ID and session ID.
- Requests are rejected if device is unverified or attendance window is inactive.

#### C) BLE Broadcasting and Scanning

- Student app broadcasts UUID via BLE for 60 seconds.
- Instructor app scans and filters devices using  $RSSI \leq -75$  dBm.
- Valid UUIDs are collected within the scanning window.

#### D) Batch Submission and Validation

- Instructor app submits all UUIDs in a single batch.
- System validates tokens (existence, expiry, usage).
- Valid students marked PRESENT; others ABSENT; tokens marked used.

#### E) Academic Management

- Admins define programs, subjects, and assign instructors.
- Students are auto-enrolled based on program and semester.
- Duplicate enrollments are prevented.

#### F) Reporting

- Instructors view sessions and attendance per course.
- Students view enrolled subjects and session-wise attendance.
- System calculates overall attendance percentage per subject.

#### G) Device Change Management

- System detects login from new devices and locks attendance.
- Students can request device change.

- Admin approves/rejects and updates device ID accordingly.

### 3.2 Technical Issues

- **BLE Limitation:** Limited 31-byte packet size; solved by broadcasting only a 16-byte UUID.
- **Network Dependency:** Internet required for token generation; no network = no attendance.
- **Signal Variability:** RSSI fluctuations handled using a -75 dBm threshold and manual verification.
- **Security & Load Handling:** One-time tokens prevent replay attacks; server designed to handle high concurrent requests.

### 3.3 Non-Functional Requirements

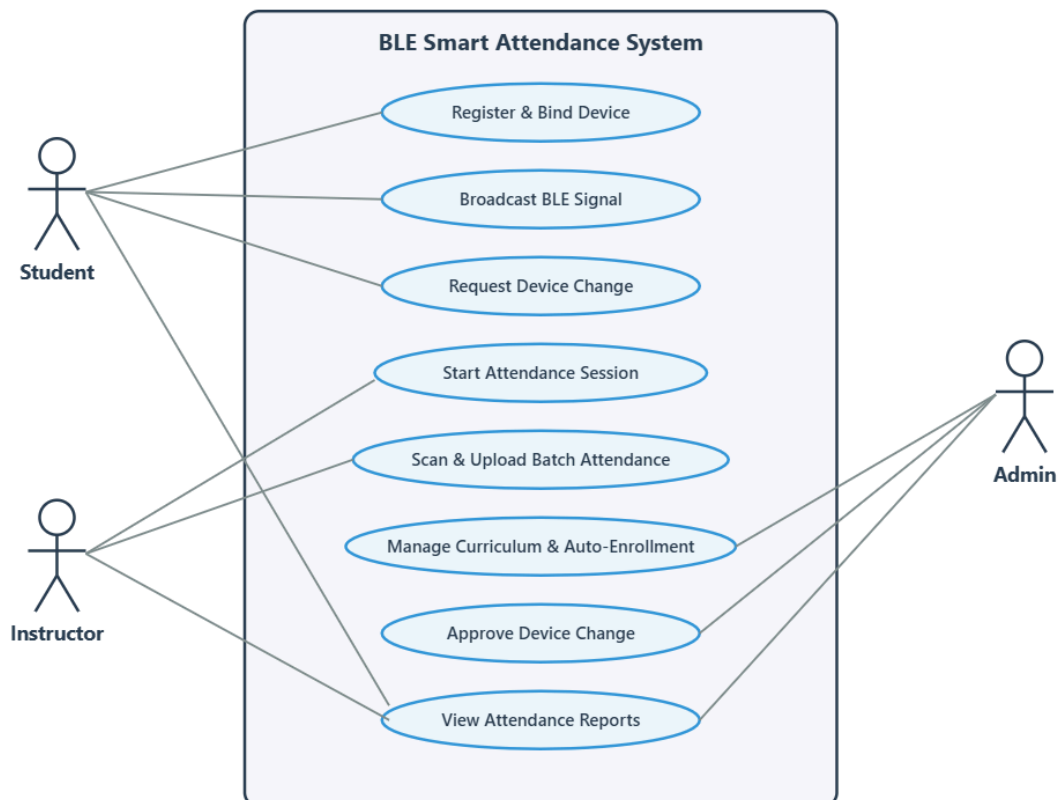
- **Security:** Passwords are securely hashed (bcrypt), all communication uses HTTPS, and JWT-based authentication ensures secure access. BLE transmits only temporary UUIDs with no sensitive data.
- **Performance:** Token generation < 500 ms, batch processing (300 students) < 3 sec, API responses < 1 sec, and supports 300+ concurrent requests.
- **Reliability:** System uptime ≥ 99.5%, BLE detection accuracy > 95% within range, false positives < 2%, and zero data loss with local caching.
- **Compatibility & Scalability:** Supports Android 8+, iOS 13+, BLE 4.0+, Node.js 18+, PostgreSQL 14+, and scales up to 500 students/class, 50 concurrent sessions, and 10,000+ records.

## 4. Design

### 4.1 Use Case Diagram

The use case diagram shows the interactions between the three actors - Student, Instructor, and Administrator - and the actions each can perform within the BLE Smart Attendance System.

#### Use Case Diagram

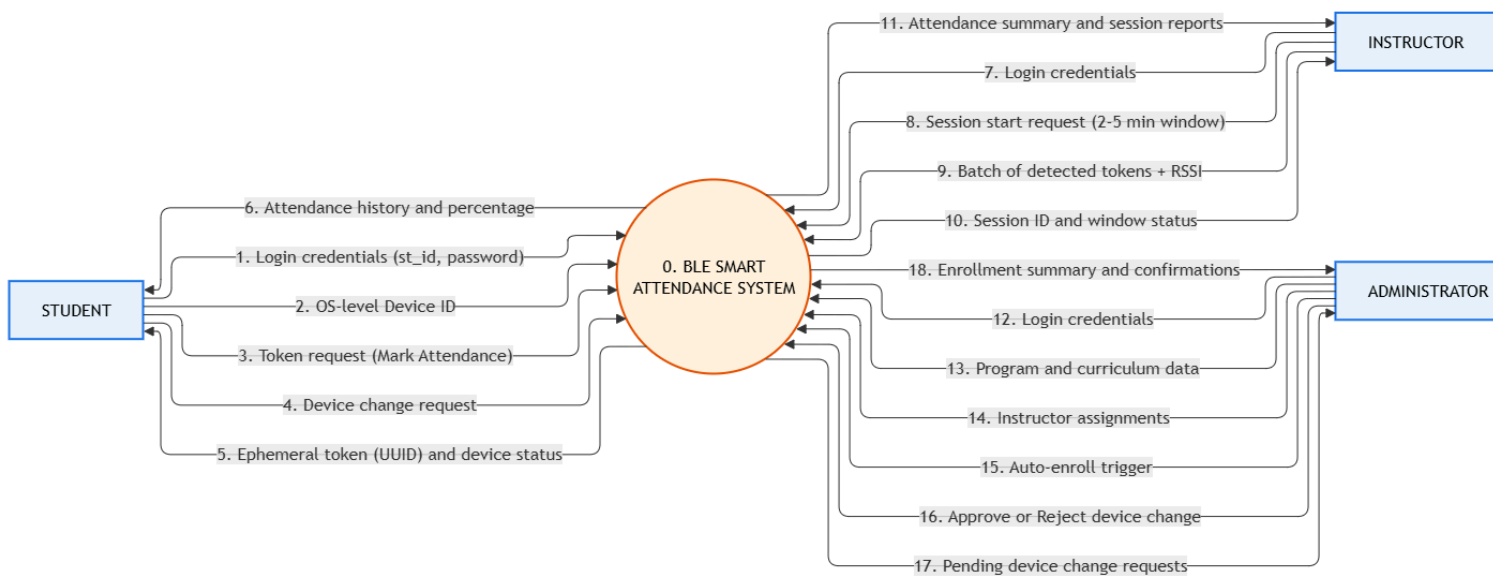


### 4.2 Data Flow Diagrams (DFD 0 ,DFD 1 and DFD 2)

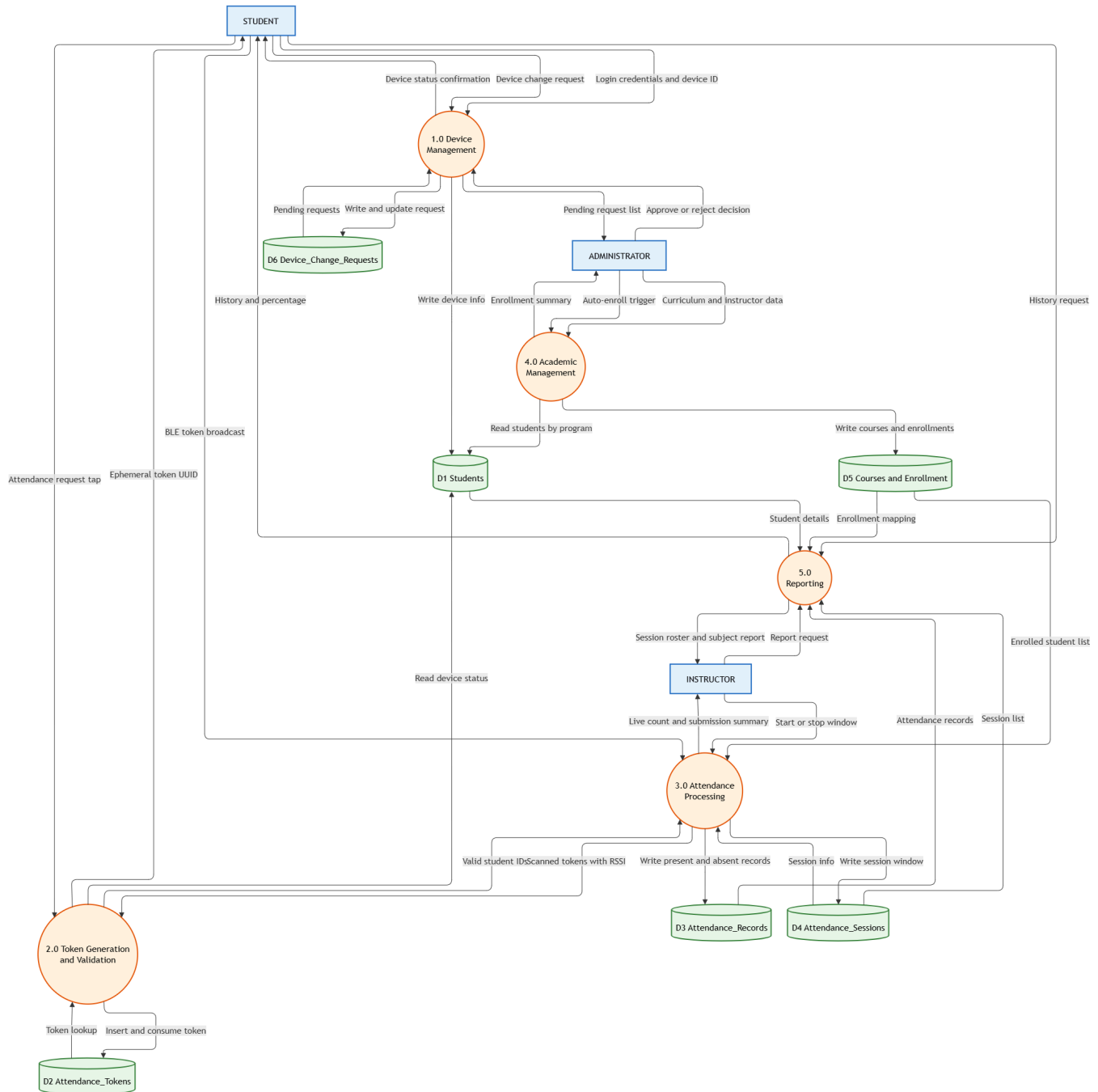
The DFD 0 (context level) shows the whole Smart Attendance System as a single process with the external entities Student, Instructor, and Administrator. The DFD 1 breaks the system into its main sub-processes - Device Management, Token

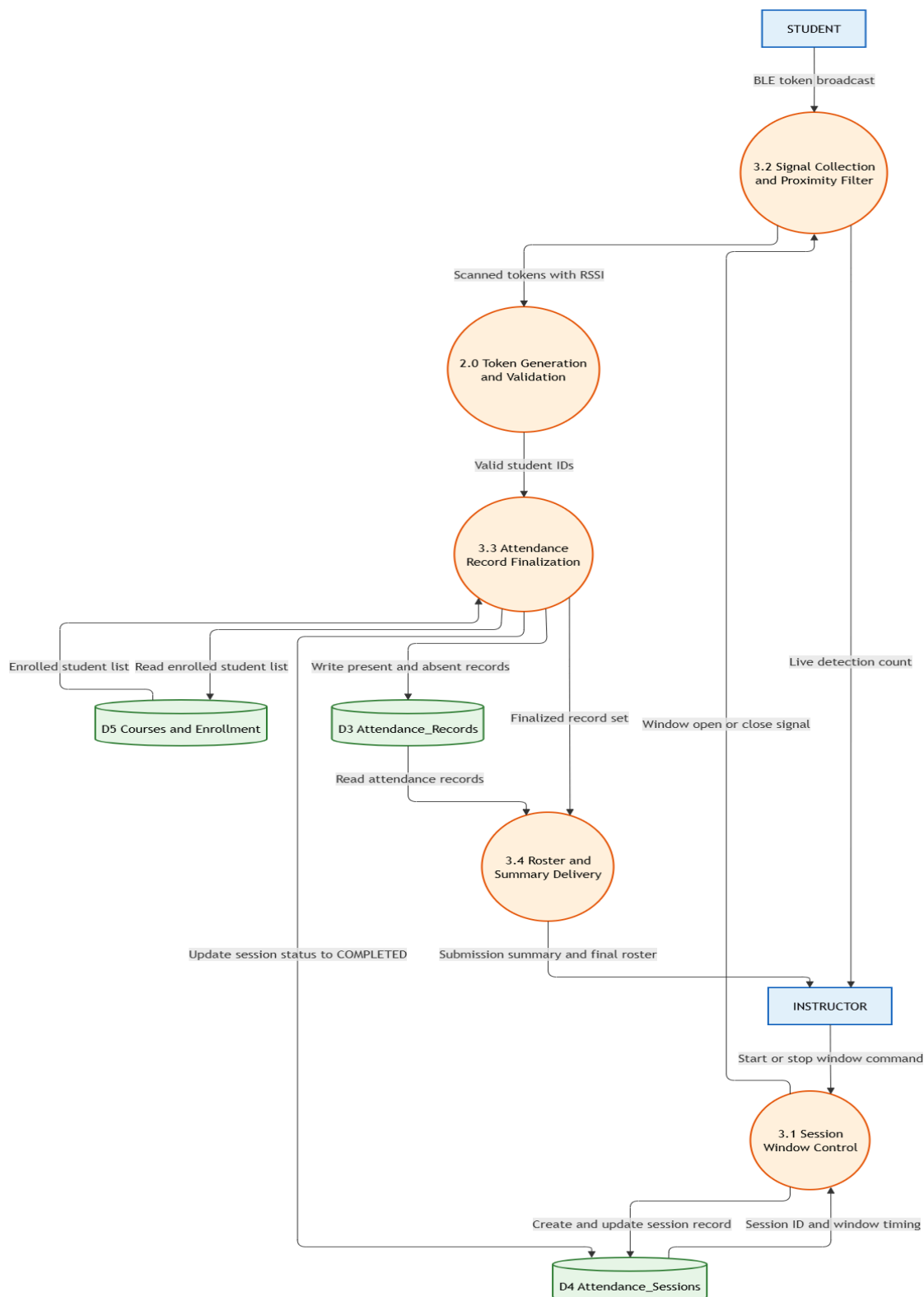
Generation and Validation, Attendance Processing, Academic Management, and Reporting - showing the data flow between them and the data stores.

## DFD 0 - Context Level



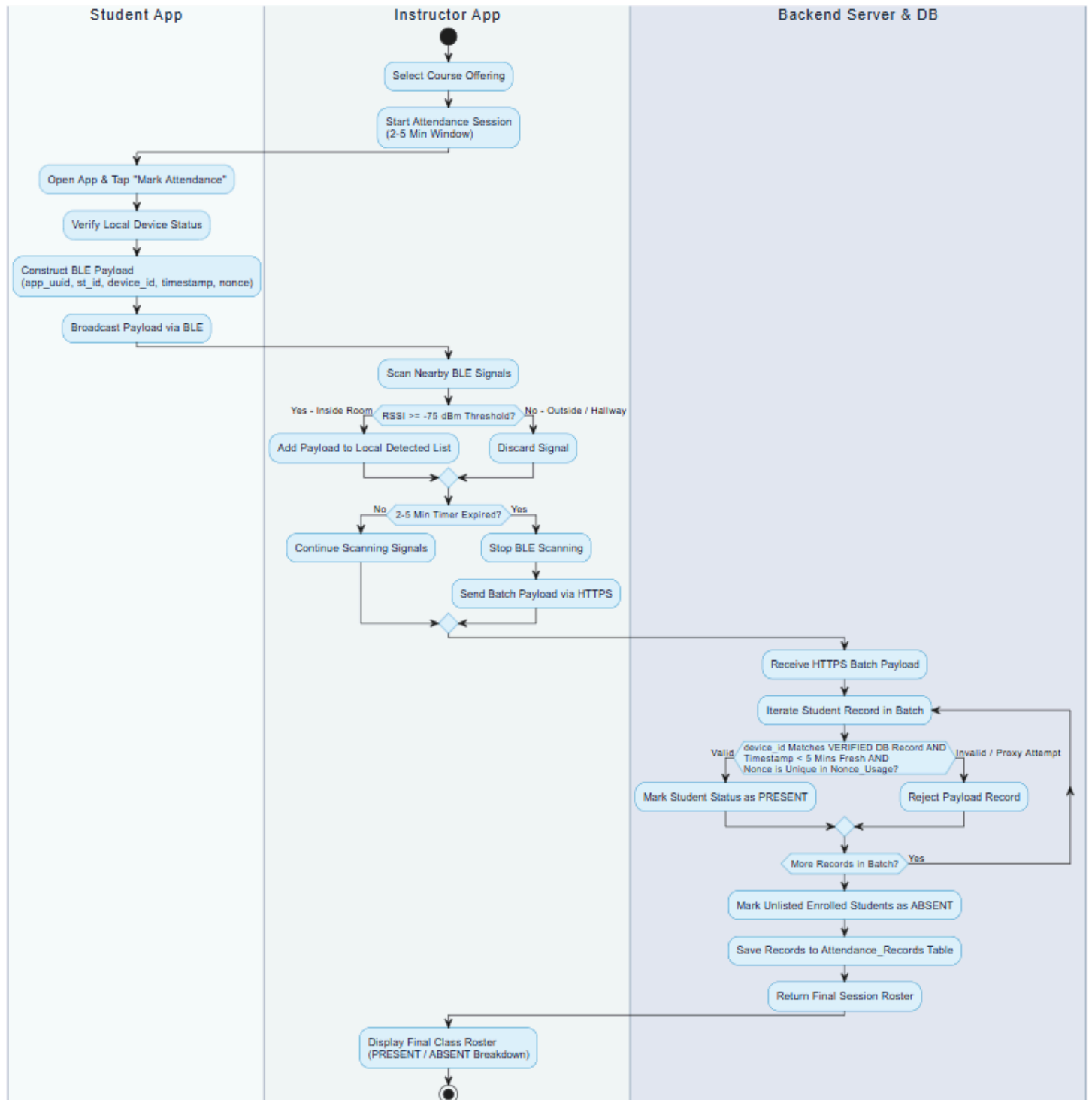
## DFD 1 - Level 1





**DFD level 2** of process , Attendance Processing

## 4.3 Activity Diagram



## 4.4 Class-Diagram

# 5 . Database Design

## 5.1 Entity-RelationShip Diagram (ERD)